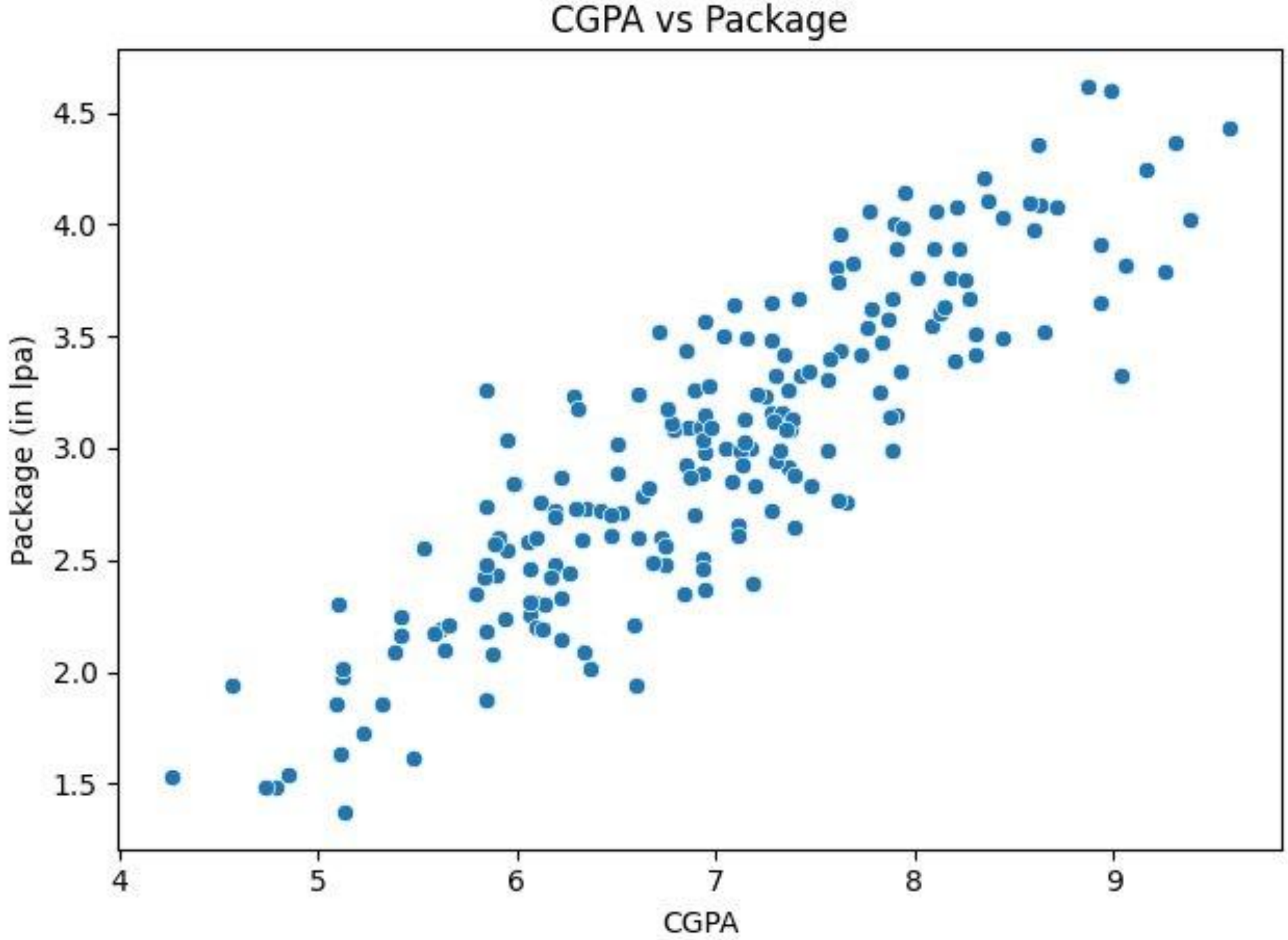
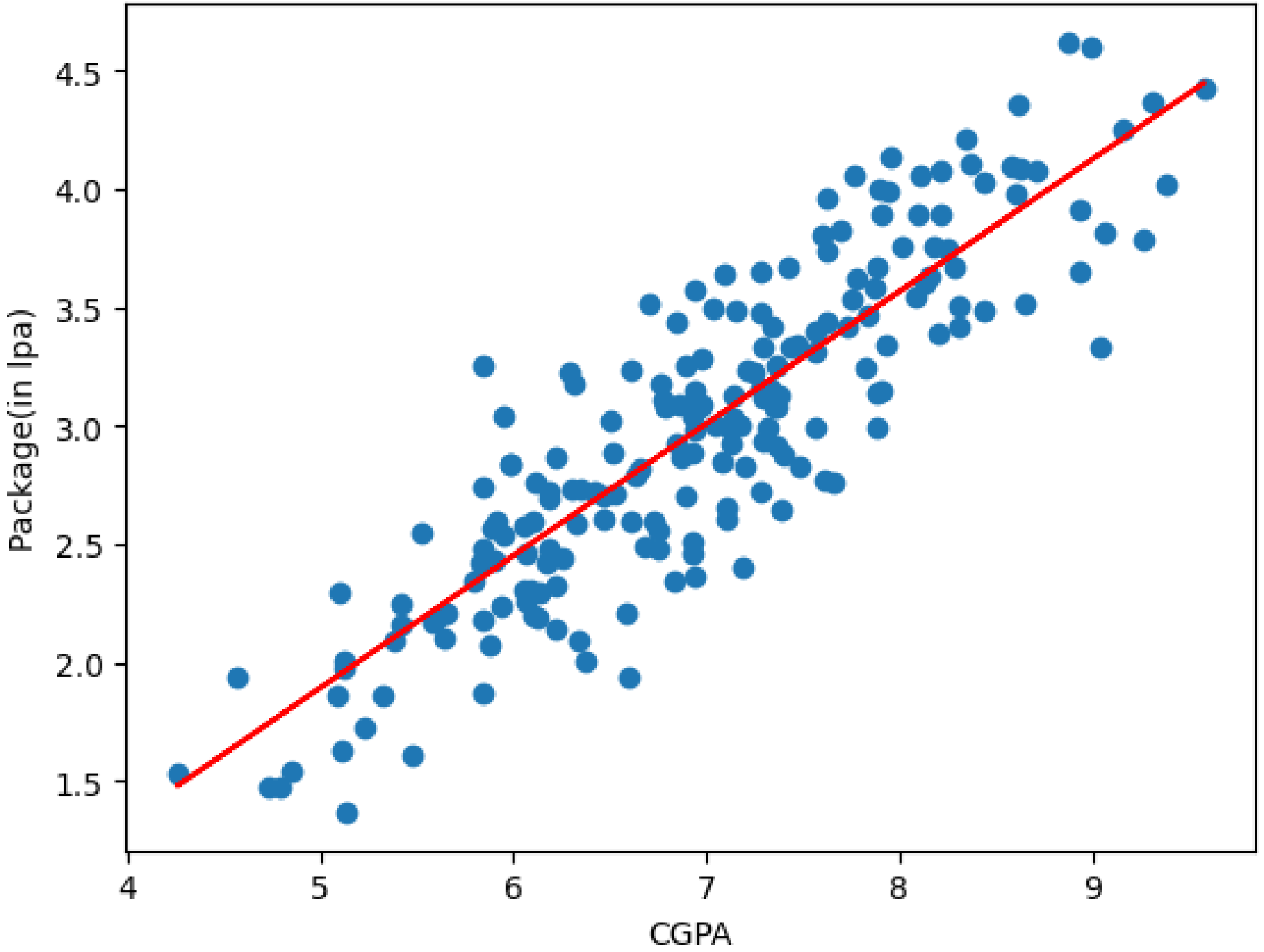

CGPA vs Package
Package (in lpa)
4.5
4.0
3.5
3.0
2.5
2.0
1.5
4
5
6
7
8
9
CGPA


Package(in lpa)
4.5
4.0
3.5
3.0
2.5
2.0
1.5
4
5
6
7
8
9
CGPA

Mathematics

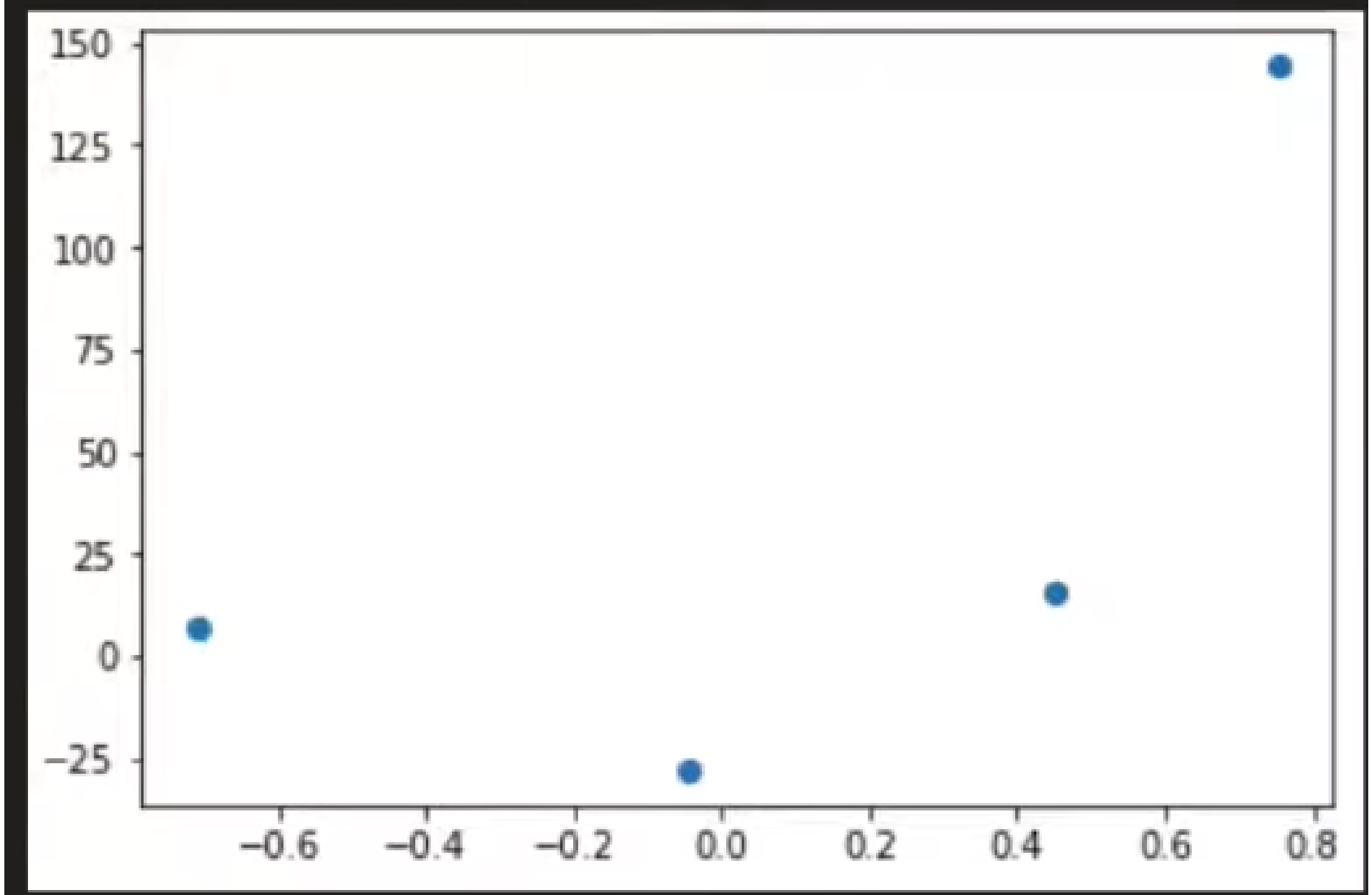
- To Calculate the best m, b we have two different solutions –

1. Closed Form Solution – Direct Formula (Ordinary Least Squares)
2. Non-Closed Form Solution - Gradient Descent for higher dimensional data.

Gradient Descent

Gradient Descent

- Gradient Descent is an iterative optimization algorithm for finding a local minimum in a differentiable function.
- A general algorithm also used in logistic regression, tsync clustering, deep learning.
- It is optimization technique on which if we provide differentiable function then it provides the minimum of the function.
- Types of Gradient Descent - Batch GD, Stochastic GD, Mini-batch GD


150
125
100
75
50
25
0
−25
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6
0.8

Gradient Descent

- We know, $E = \sum(yi - mxi - b)^2$
- Assume, we know m=78.35 then we get
- $E = \sum(yi - 78.35\ xi - b)^2$
- Now, our goal is to find such b so that E is minimum.
- We want to replace ordinary least squares (OLS) with Gradient Descent.

Step By Step process

1. Select a random b.

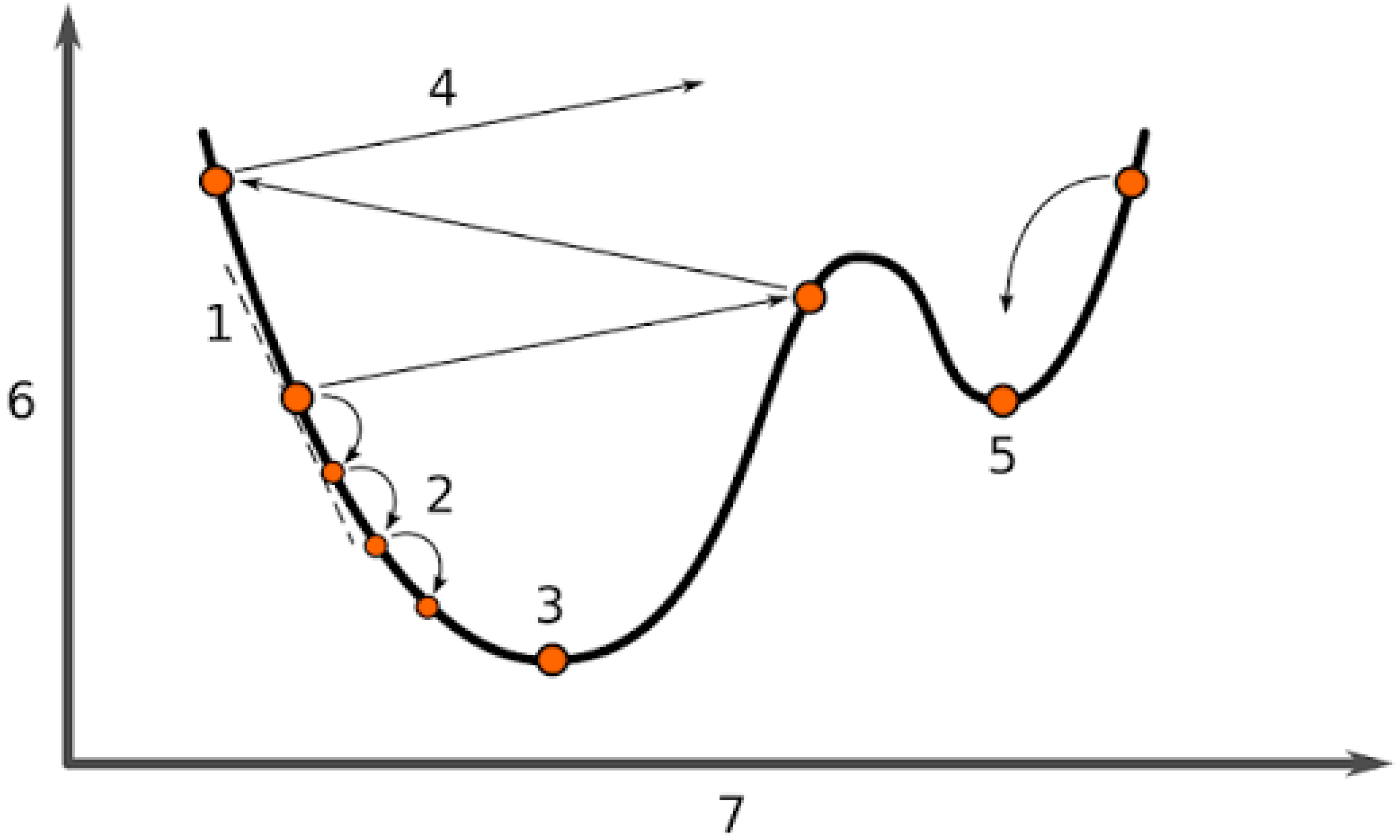
2. Find slope of the random value. Ex: for function=x^2+2 where x=5 we calculate slope of the curve using, dy/dx =2x = 2*5=10

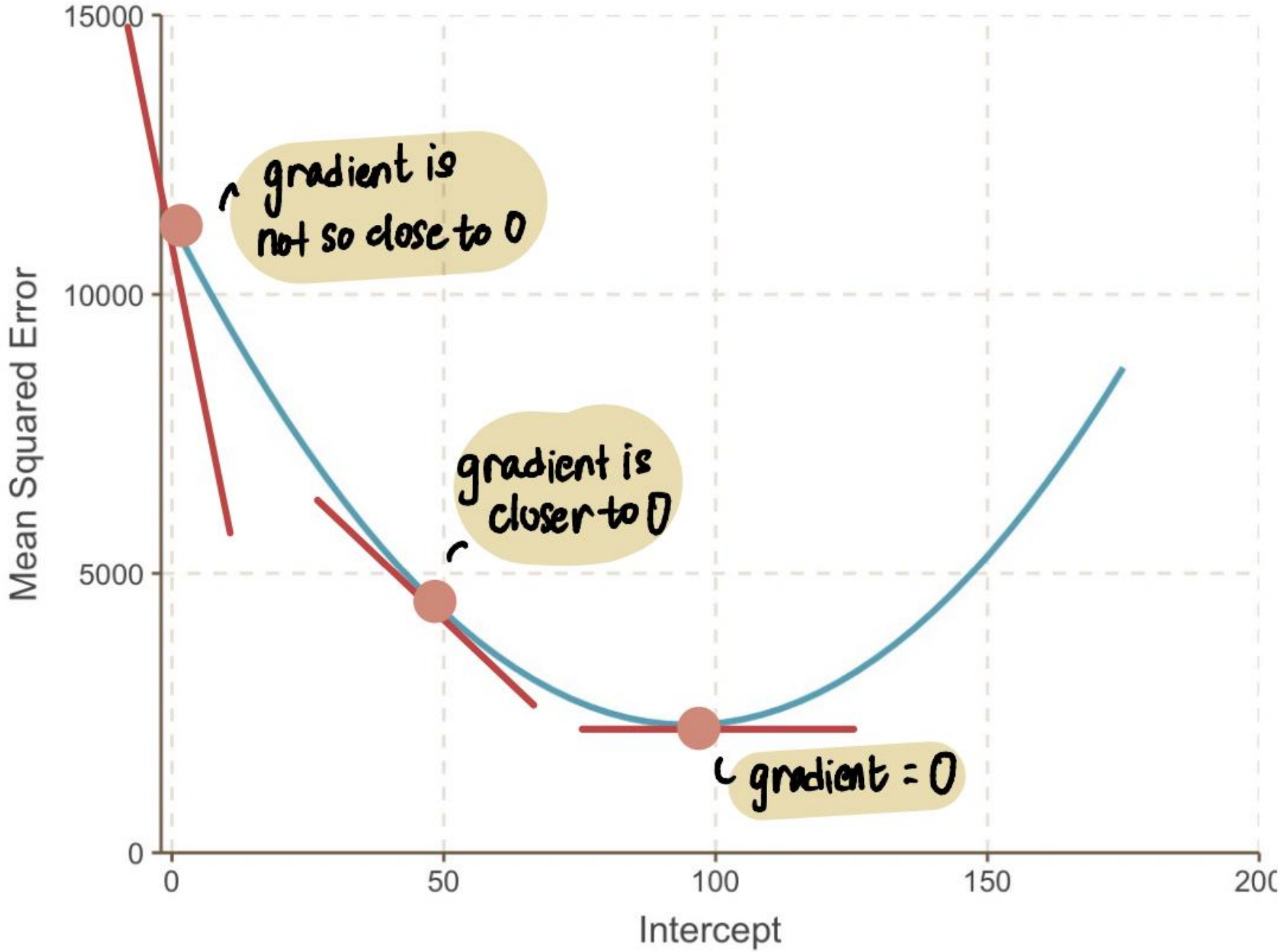
   If slope negative go forward (increment b) else vice versa.
   We get equation such as, Bnew = Bold – Slope

3. Final Equation = Bnew = Bold – μ * Slope where μ = learning rate

4. Stop when Bnew – Bold is 0.0001 or closer to 0 or we can give Iteration limit. (epochs)

- Bnew = Bold – Slope
- Suppose, b=-10, slope=-50, we get Bnew = -10-(-50) = 40
- Suppose, b=10, slope=50, we get Bnew = 10-50 = -40
- Issue is how to stop zigzag? By using a learning rate (μ =0.01/0.001/0.1).
- For b=-10, slope=-50, we get Bnew = -10-(0.01*-50) = -10+0.5 = -9.5
- Now our b=-9.5, slope assume -40, Bnew = -9.5-(0.01*-40) = -9.1
- Now our b=-9.1, slope assume -30, Bnew = -9.1-(0.01*-30) = -8.8
- We stop the searching when Bnew-Bold = closer to 0 (ex: 0.0001 etc.)


4
1
6
2
3
5
7


15000
10000
5000
0
Mean Squared Error
0
50
100
150
200
Intercept
gradient is not so close to 0
gradient is closer to 0
gradient = 0

Mathematical Formulation of Gradient Descent

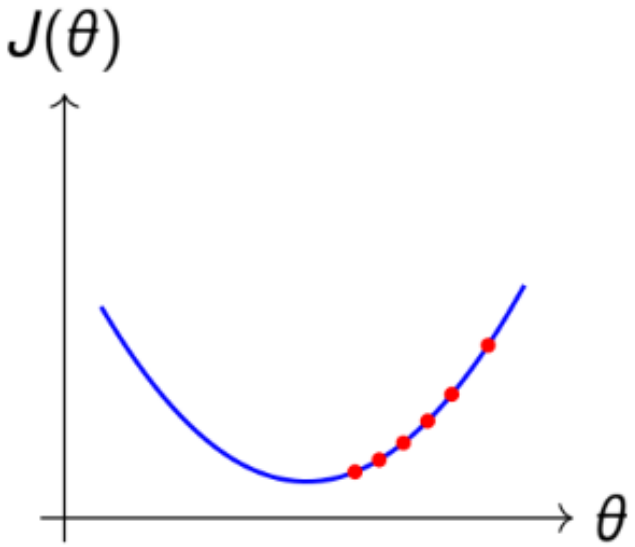
- Start with a random b, assume b=0
- For i in epoch 100:
  - Update b
  - Bnew = Bold – μ * Slope where μ = 0.01

- Find slope at b from cost function = $\sum(yi - \hat{y})^2$
- dE / db = d / db (E)
- dE / db = d / db ($\sum(yi – (mxi-b))^2$)
- 2 ($\sum(yi – (mxi-b))$ * d / db (yi – (mxi-b)) = 2 ($\sum(yi – (mxi-b))$ * -1
- -2 ($\sum(yi – (mxi-b))$ [Equation of slope]

Effect of Learning Rate α

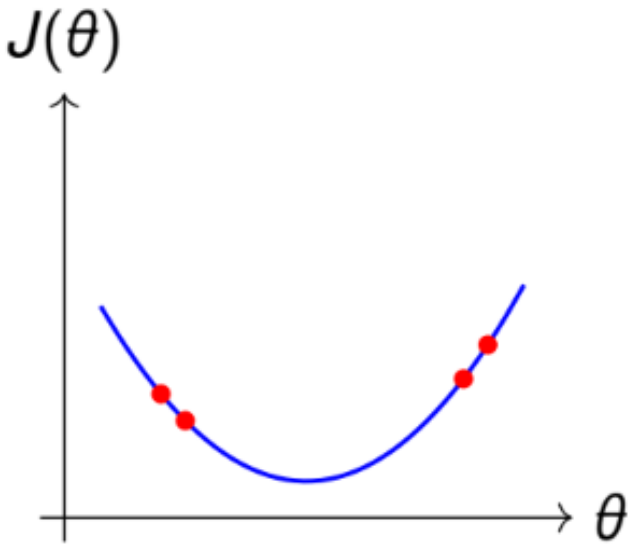
Too Small α

- Very slow convergence
- Many iterations required

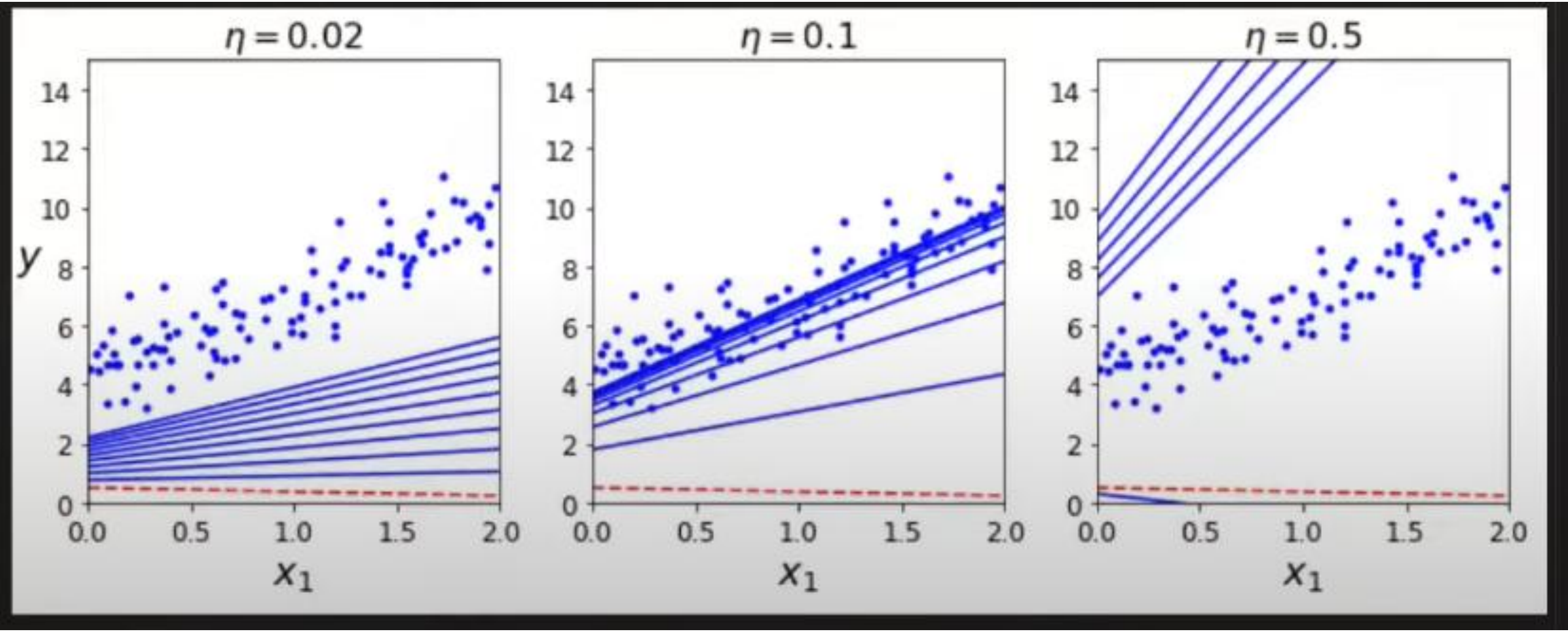


Too Large α

- Oscillation
- May diverge



Choose α carefully for stable and fast convergence.


$\eta = 0.02$
$\eta = 0.1$
$\eta = 0.5$
y
x_1
x_1
x_1
0
2
4
6
8
10
12
14
0.0
0.5
1.0
1.5
2.0

Versatility of Gradient Descent

- Gradient Descent can be applicable on not just linear regression but other methods such as logistic, deep learning based on changing the loss function to calculate slope accordingly.

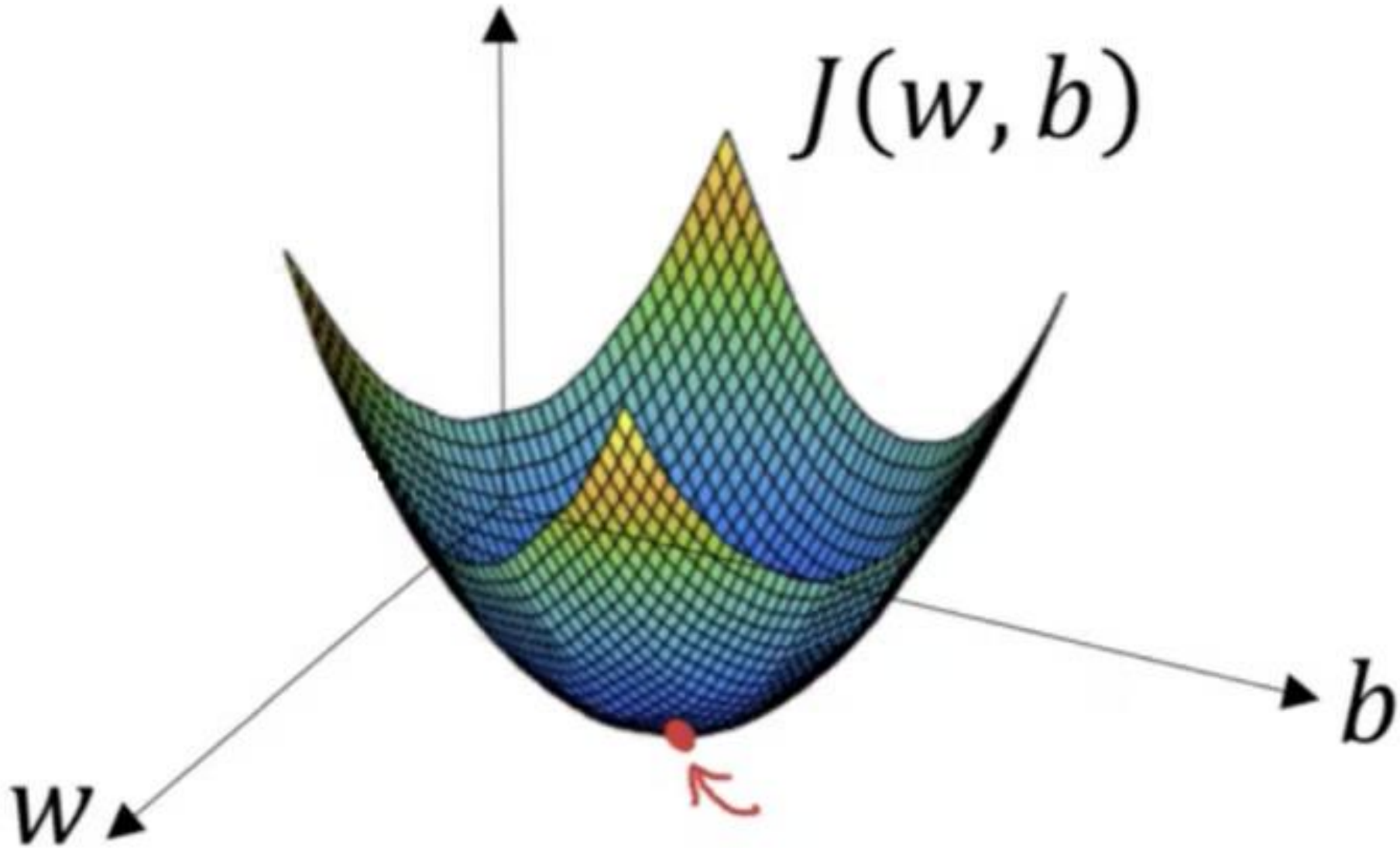
- b_new = b_old – μ * slope [Equation is independent of the ml algorithm]

- Slope = DE/Db where E, loss function varies based on approaches.

Mathematical Formulation of Gradient Descent

- Start with a random m, and b, assume m=1, b=0
- For i in epoch 100:
  - Update both m, b where $\mu = 0.01$
  - Bnew = Bold – μ * Slope
  - Mnew = Mold - μ * Slope

- Find slope at m,b from cost function = $\sum(yi - \hat{y})^2 = (\sum(yi - (mxi-b))^2)$
- $dE / db = -2 (\sum(yi - (mxi-b))$
- $dE / dm = -2 (\sum(yi - (mxi-b)) * xi$


$J(w,b)$
w
b